

Проектирование микропроцессорной системы на базе МК i8051

Описание программной части

Листов 29

Инд. № подл.	Подп. и дата
Взам. инд. №	Инд. № дубл.
Подп. и дата	Подп. и дата

Выполнил: Грошев А.В.
Группа: А-06-23
Вариант: 1

Проверил: Ключанский А.А.

Москва, 2026

АННОТАЦИЯ

В данном документе приведено описание программы «Программа управления объектом», предназначенной для управления объектом микропроцессорной системой (МПС) на базе микроконтроллера AT89S53: ввода данных с клавиатуры и от внешнего устройства, формирования управляющих сигналов Y_1 и Y_2 и вывода информации на семисегментный индикатор.

Для функционирования данной программы необходима МПС, собранная по схеме электрической принципиальной варианта 1 (пояснительная записка по аппаратной части).

Исходным языком программы «Программа управления объектом» является ассемблер микроконтроллеров семейства MCS-51.

Основными функциями программы являются: инициализация МПС, сканирование клавиатуры, работа с кольцевым буфером на 1210 байт, выработка stroba Y_1 длительностью 176 мс, выработка сигнала Y_2 со стробом 1536 мкс, индикация на одном разряде.

Оформление программного документа «Описание программы» произведено по требованиям ЕСПД (ГОСТ 19.101-77, ГОСТ 19.103-77, ГОСТ 19.104-78, ГОСТ 19.105-78, ГОСТ 19.106-78, ГОСТ 19.402-78, ГОСТ 19.604-78).

СОДЕРЖАНИЕ

1 Общие сведения.....	4
2 Функциональное назначение.....	5
2.1 Классы решаемых задач.....	5
2.2 Назначение программы.....	5
2.3 Сведения о функциональных ограничениях на применение.....	5
3 Описание логической структуры.....	6
3.1 Алгоритм программы.....	6
3.2 Структура программы с описанием функций составных частей и связи между ними	19
3.3 Связи программы с другими программами.....	20
4 Входные данные.....	21
4.1 Клавиатура.....	21
4.2 Сигнал X2.....	21
5 Выходные данные.....	22
5.1 Характер и организация выходных данных.....	22
5.2 Формат, описание и способ кодирования выходных данных.....	22
ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ.....	24
ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ.....	25
ПРИЛОЖЕНИЕ А. Листинги программ.....	26

1 Общие сведения

Обозначение и наименование программы — «Программа управления объектом», вариант 1, группа А-06-23.

Программа низкого уровня не требует для своей работы иных программ. Исходным языком программирования является ассемблер MCS-51 (синтаксис Intel ASM51, совместимый с Keil A51 и MCU 8051 IDE).

Среда разработки и инструментальные средства: интегрированная среда Keil μ Vision V5 (Keil Software, пакет PK51); ассемблер — A51; компоновщик — BL51.

2 Функциональное назначение

2.1 Классы решаемых задач

- инициализация МПС;
- сканирование клавиатуры — ввод сигнала X_1 (число, введённое с клавиатуры);
- формирование сигнала Y_1 (строб заданной длительности);
- ввод сигнала X_2 (сохранение данных в кольцевом буфере);
- формирование сигнала Y_2 и сопровождающего строба;
- вывод символа на индикатор.

2.2 Назначение программы

МПС принимает информацию $\{X\}$ об объекте управления от датчиков, вырабатывает управляющие воздействия $\{Y\}$ в соответствии с заданным алгоритмом управления и подаёт их на исполнительные механизмы. Программа разбита на подзадачи по функциональному признаку:

- инициализация МПС;
- обработка прерываний: опрос клавиатуры, запись в буфер, таймеры генераторов сигналов Y ;
- определение кода нажатой клавиши;
- вывод данных на семисегментный индикатор;
- формирование сигнала Y_2 ;
- обеспечение работы кольцевого буфера (запись и чтение отсчётов X_2).

В соответствии с вариантом (табл. 1 ТЗ) разработан код трёх программ:

- программа 1 — обработчик прерывания клавиатуры (сканирование матрицы), код нажатой клавиши в аккумуляторе или 0FFh;
- программа 2 — запись в кольцевой буфер (данные через аккумулятор) с управлением флагами пустоты и переполнения;
- программа 3 — выработка строба Y_1 длительностью T_1 (прерывание таймера).

2.3 Сведения о функциональных ограничениях на применение

В качестве основы для построения МПС выбран микроконтроллер семейства MCS-51 — AT89S53, тактовая частота 12 МГц (машинный цикл 1 мкс). Таймер 2 и сторожевой таймер не используются (совместимость с i8051).

Интерфейсы:

- входные данные X_2 — 8-разрядное целое без знака по независимой параллельной шине со стробом записи низкого уровня в двухпортовое ОЗУ, нерегулярно, до 4 отсчётов в секунду;
- входные данные X_1 — десятичная цифра 0–9 с матричной клавиатуры 3×4 (сканирование по столбцам);
- выходной сигнал Y_2 — 8-разрядный параллельный код в выходном регистре со стробом низкого уровня длительностью 1536 мкс на выводе P1.2;
- выходной сигнал Y_1 — строб низкого уровня длительностью 176 мс на выводе P1.1;
- индикация — один семисегментный индикатор (общий катод), время индикации символа не более $T_3 = 2,150$ с.

3 Описание логической структуры

3.1 Алгоритм программы

После сброса МК выполняет инициализацию (рисунок 1): ставит стек на 07h, разрешает дешифратор адреса ($CS_EN = 1$), переводит стробы Y1 и Y2 в пассивную единицу, опускает все столбцы клавиатуры и гасит индикатор, объявляет буфер пустым, настраивает Timer0 и Timer1 в режим 1, а INT0 и INT1 — на срабатывание по спаду, и только после этого разрешает прерывания. Основная программа (рисунок 2) в цикле смотрит, есть ли что выдавать: если $X1 \neq 0$, в буфере есть отсчёт и предыдущий строб Y2 уже закончился, она читает отсчёт из «хвоста» и выдаёт $Y2 = (7 + X1 + X2) \bmod 256$ со стробом.

Всё, что связано со временем и с внешними событиями, сделано в прерываниях. INT0 приходит от клавиатуры: обработчик опрашивает матрицу (рисунок 3) и по коду клавиши меняет X1, запускает строб Y1 или очищает буфер (рисунок 4). INT1 — строб внешнего устройства: обработчик записывает отсчёт X2 в ячейку «головы» (рисунки 5 и 6). Timer0 тикает каждые 2 мс и отсчитывает длительность строба Y1 и время индикации T3 (рисунок 10), Timer1 одним интервалом отмеряет строб Y2 1536 мкс и поднимает его (рисунок 13). Благодаря этому ни одна процедура не ждёт в цикле, и основная программа всегда готова обработать следующий отсчёт.

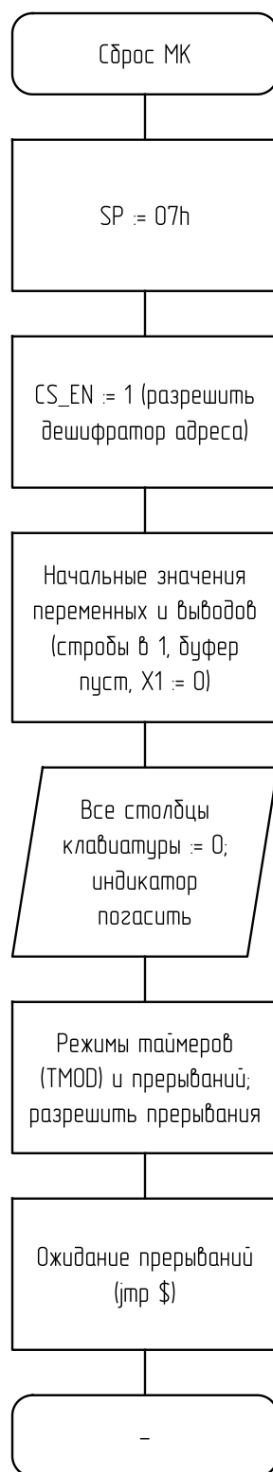


Рисунок 1 — Схема алгоритма: Инициализация МПС и основная программа

Схема «Инициализация МПС и основная программа» выполняется так: вход — «Сброс МК»; $SP := 07h$; $CS_EN := 1$ (разрешить дешифратор адреса); начальные значения переменных и выводов (стробы в 1, буфер пуст, $X1 := 0$); обмен с устройством: все столбцы клавиатуры := 0; индикатор погасить; режимы таймеров (TMOD) и прерываний; разрешить прерывания; ожидание прерываний (jmp \$).

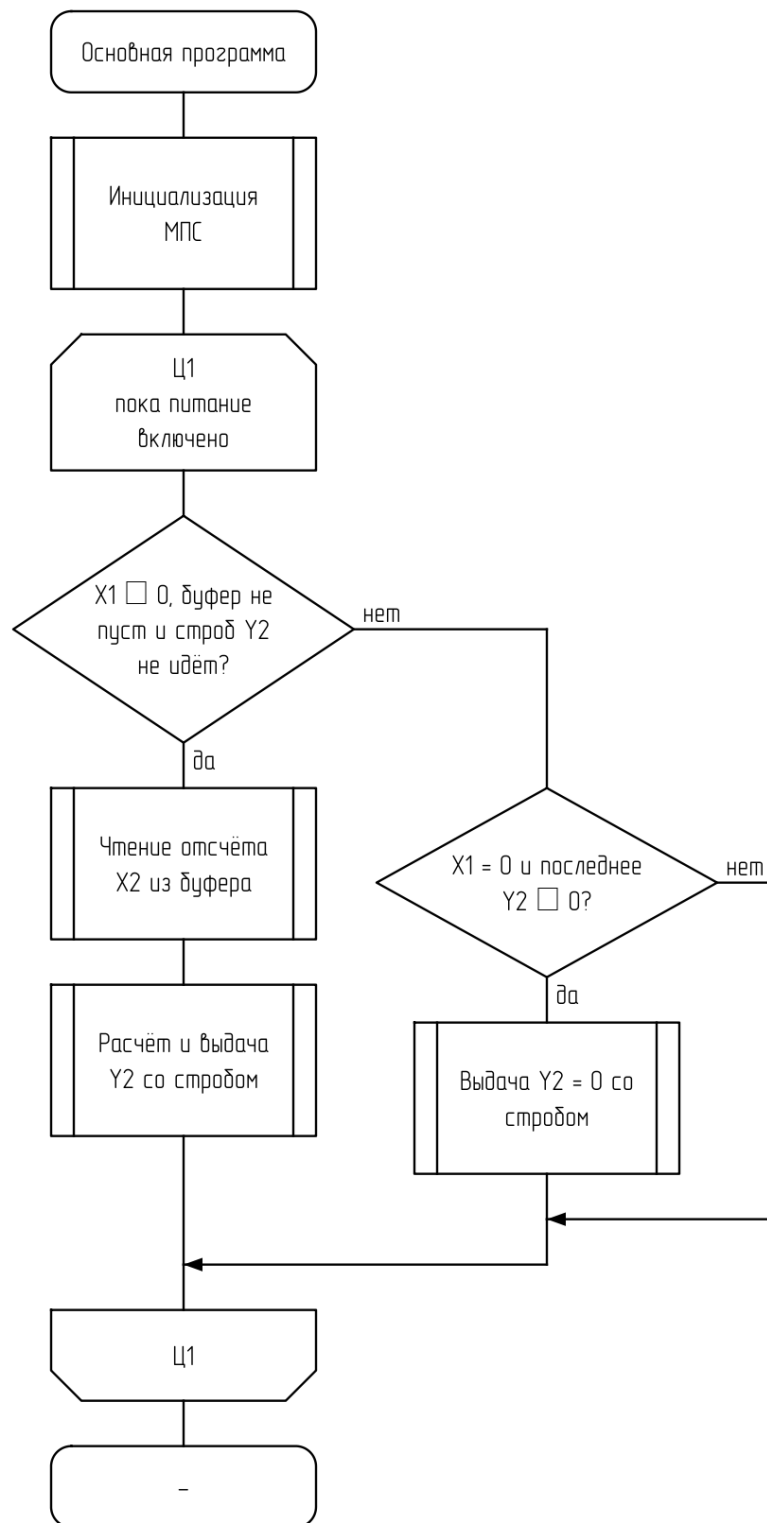


Рисунок 2 — Схема алгоритма: Основной цикл программы

Схема «Основной цикл программы» выполняется так: вход — «Основная программа»; вызывается подпрограмма: инициализация МПС; в цикле Ц1 (пока питание включено) повторяется: проверяется условие « $X1 \neq 0$, буфер не пуст и строб Y2 не идёт»: если да — вызывается подпрограмма: чтение отсчёта X2 из буфера, вызывается подпрограмма: расчёт и выдача Y2 со стробом, иначе — проверяется условие « $X1 = 0$ и последнее $Y2 \neq 0$ »: если да — вызывается подпрограмма: выдача Y2 = 0 со стробом.

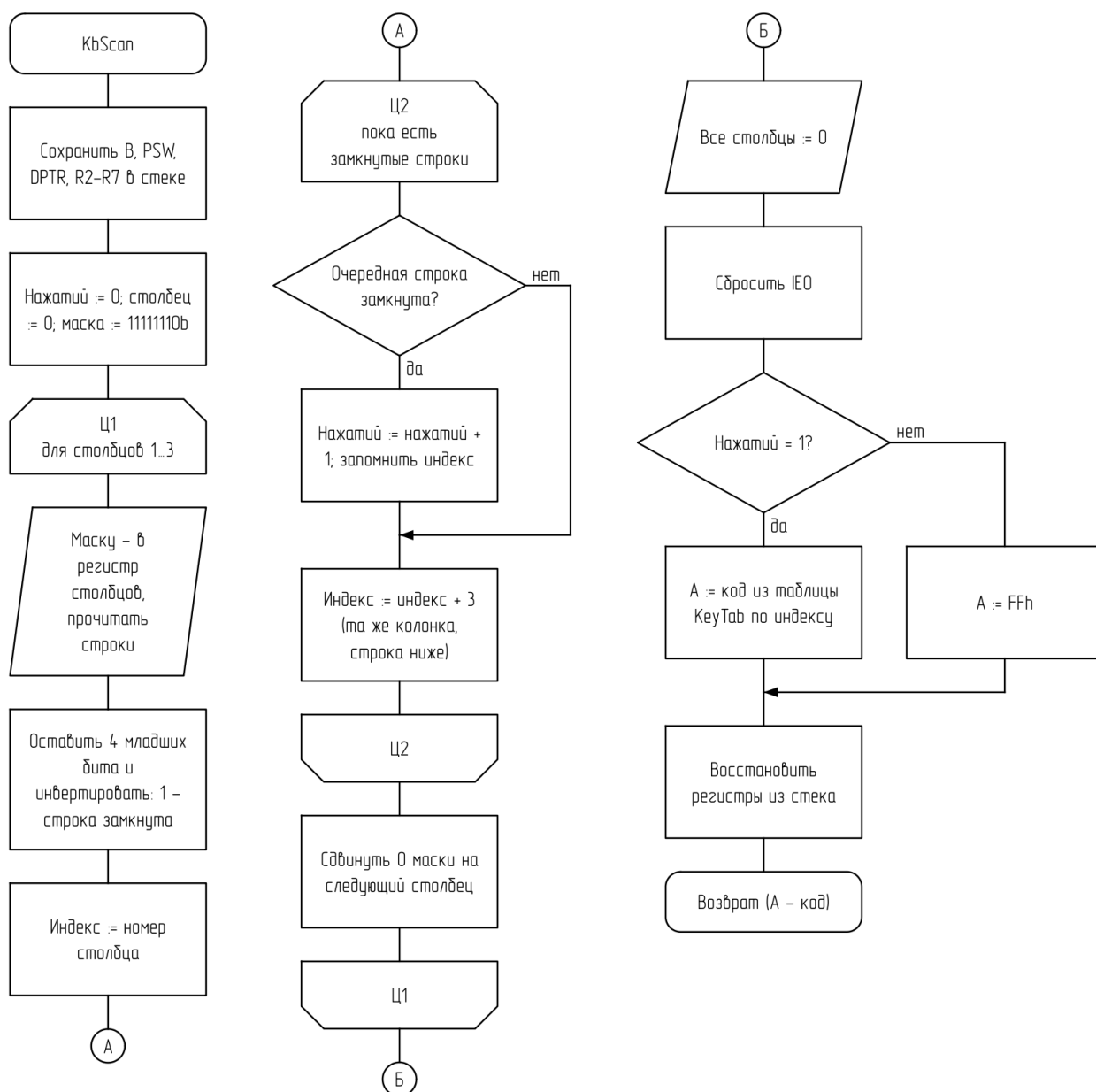


Рисунок 3 — Схема алгоритма: Опрос клавиатуры (обработчик INT0)

Процедура KbScan по очереди опускает в 0 один из трёх столбцов и читает четыре строки. Старшие биты прочитанного байта не подключены, поэтому они отбрасываются маской, а оставшиеся биты инвертируются, чтобы замкнутая строка давала 1. Каждая найденная единица увеличивает счётчик нажатий, а индекс клавиши считается как номер столбца плюс три на каждую строку вниз — это сразу номер элемента в таблице KeyTab. После опроса все столбцы снова опускаются, иначе следующее нажатие не вызовет INT0, и сбрасывается IEO, который успел установиться, пока линия дёргалась при переборе столбцов. Код из таблицы возвращается, только если нажата ровно одна клавиша; иначе возвращается FFh.

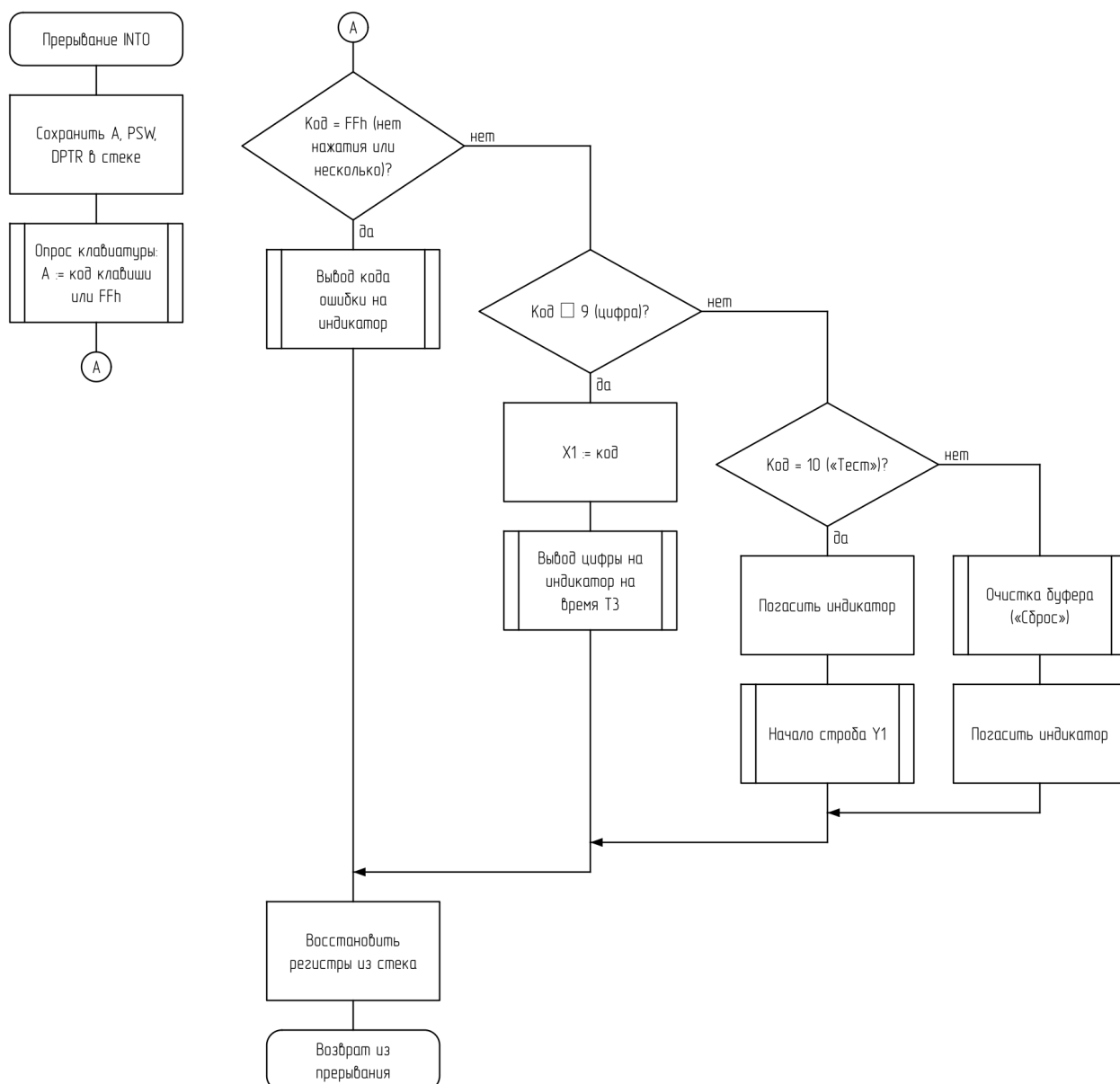


Рисунок 4 — Схема алгоритма: Обработчик прерывания INT0: реакция на клавишу

Схема «Обработчик прерывания INT0: реакция на клавишу» выполняется так: вход — «Прерывание INT0»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: опрос клавиатуры: A := код клавиши или FFh; проверяется условие «Код = FFh (нет нажатия или несколько)»: если да — вызывается подпрограмма: вывод кода ошибки на индикатор, иначе — проверяется условие «Код ≤ 9 (цифра)»: если да — X1 := код, вызывается подпрограмма: вывод цифры на индикатор на время T3, иначе — проверяется условие «Код = 10 («Тест»)»: если да — погасить индикатор, вызывается подпрограмма: начало строба Y1, иначе — вызывается подпрограмма: очистка буфера («Сброс»), погасить индикатор; восстановить регистры из стека; в конце — возврат из прерывания.

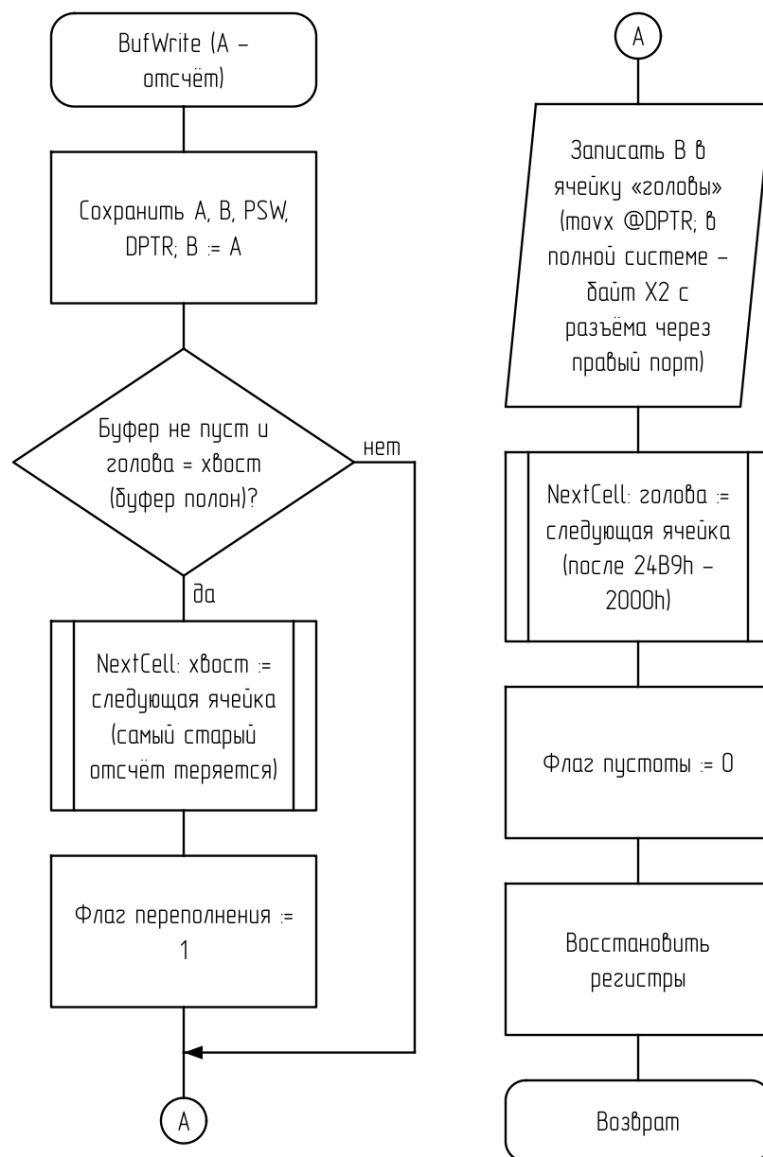


Рисунок 5 — Схема алгоритма: Запись отсчёта в кольцевой буфер

BufWrite сначала проверяет, полон ли буфер: это случай, когда буфер не пуст, а «голова» совпала с «хвостом» по обоим байтам адреса. Тогда самый старый отсчёт приносится в жертву — «хвост» сдвигается на одну ячейку, и ставится флаг переполнения. Затем отсчёт записывается по адресу «головы», «голова» переходит к следующей ячейке через общую подпрограмму NextCell (после 24B9h она возвращается на 2000h), а флаг пустоты сбрасывается. Так в буфере всегда помещается $V = 1210$ отсчётов, и при переполнении теряются самые давние данные, а не только что пришедшие.

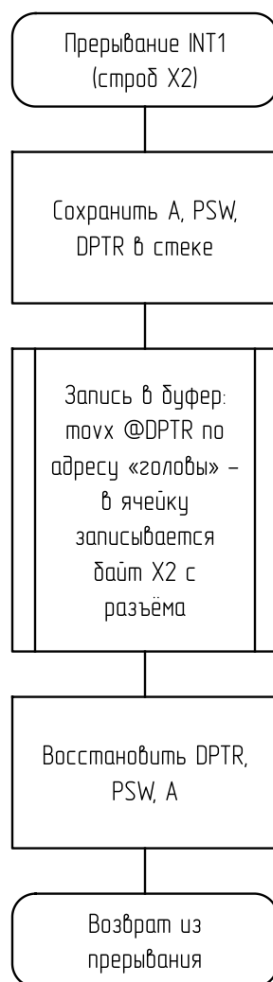


Рисунок 6 — Схема алгоритма: Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства

Схема «Обработчик прерывания INT1: приём отсчёта X2 от внешнего устройства» выполняется так: вход — «Прерывание INT1 (строб X2)»; сохранить A, PSW, DPTR в стеке; вызывается подпрограмма: запись в буфер: `movx @DPTR` по адресу «головы» — в ячейку записывается байт X2 с разъёма; восстановить DPTR, PSW, A; в конце — возврат из прерывания.

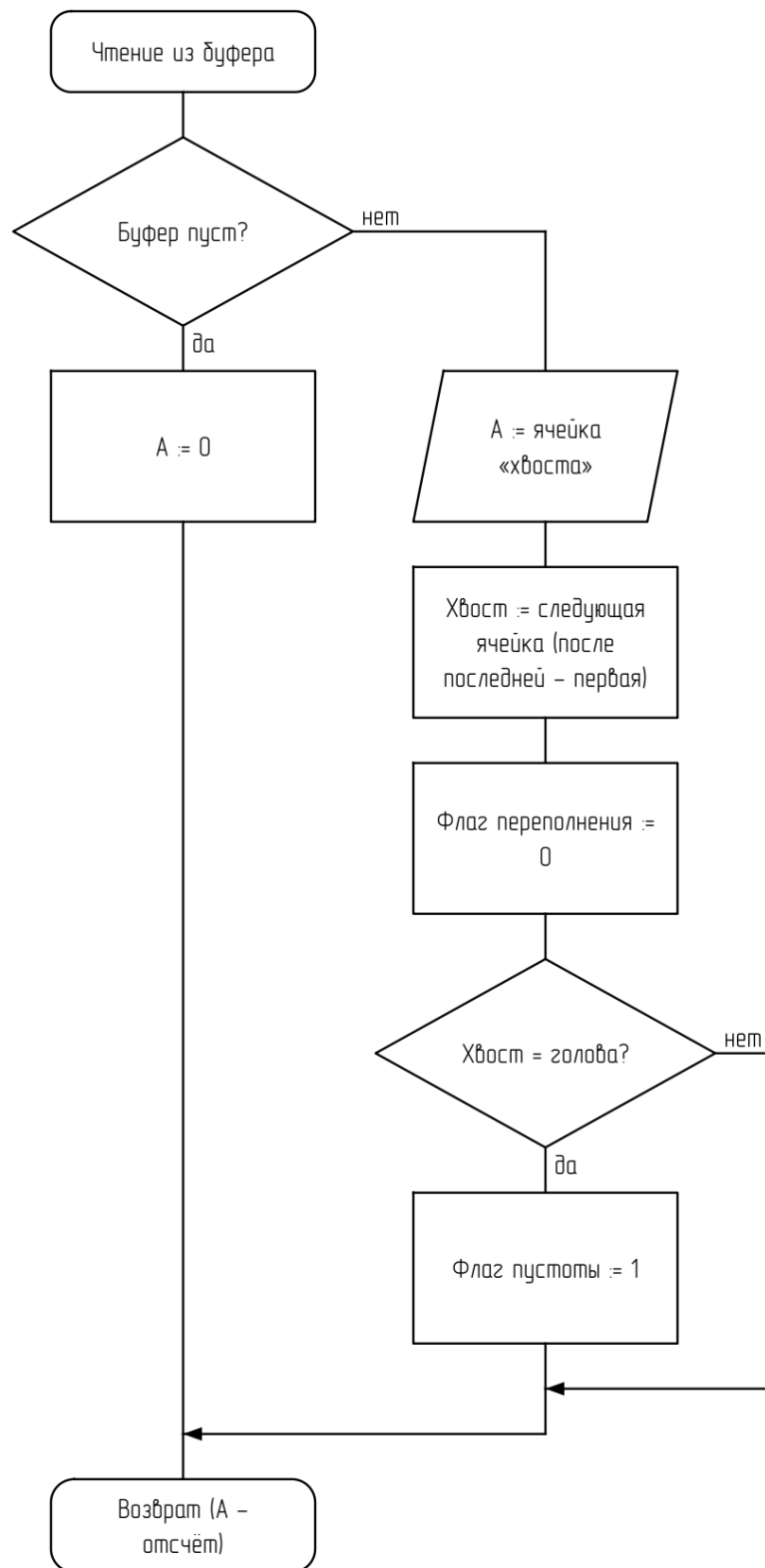


Рисунок 7 — Схема алгоритма: Чтение отсчёта из кольцевого буфера

Схема «Чтение отсчёта из кольцевого буфера» выполняется так: вход — «Чтение из буфера»; проверяется условие «Буфер пуст»: если да — $A := 0$, иначе — обмен с устройством: $A :=$ ячейка «хвоста», $\text{хвост} :=$ следующая ячейка (после последней — первая), $\text{флаг переполнения} := 0$, проверяется условие «Хвост = голова»: если да — $\text{флаг пустоты} := 1$; в конце — возврат (A — отсчёт).

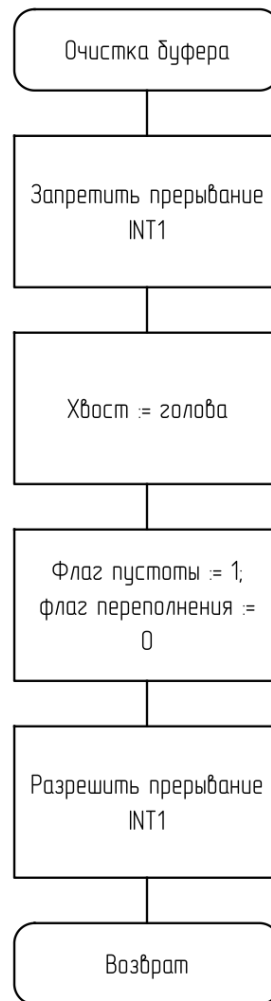


Рисунок 8 — Схема алгоритма: Очистка кольцевого буфера (клавиша «Сброс»)

Схема «Очистка кольцевого буфера (клавиша «Сброс»)» выполняется так: вход — «Очистка буфера»; запретить прерывание INT1; хвост := голова; флаг пустоты := 1; флаг переполнения := 0; разрешить прерывание INT1; в конце — возврат.

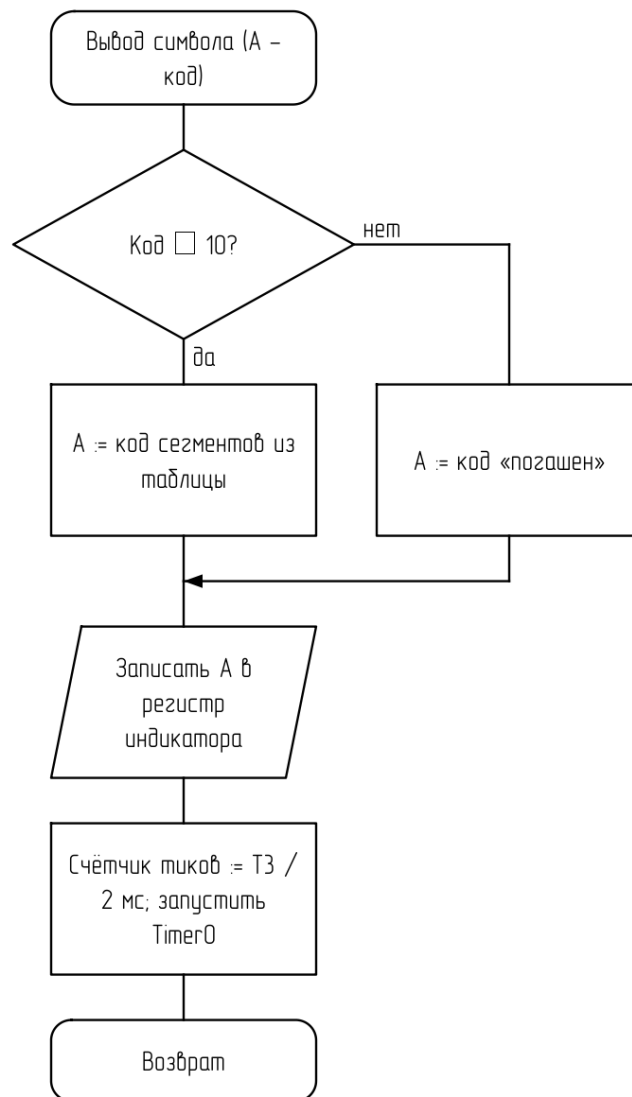


Рисунок 9 — Схема алгоритма: Вывод символа на индикатор

Схема «Вывод символа на индикатор» выполняется так: вход — «Вывод символа (A — код)»; проверяется условие «Код ≤ 10 »: если да — $A :=$ код сегментов из таблицы, иначе — $A :=$ код «погашен»; обмен с устройством: записать A в регистр индикатора; счётчик тиков $:= T3 / 2$ мс; запустить Timer0; в конце — возврат.

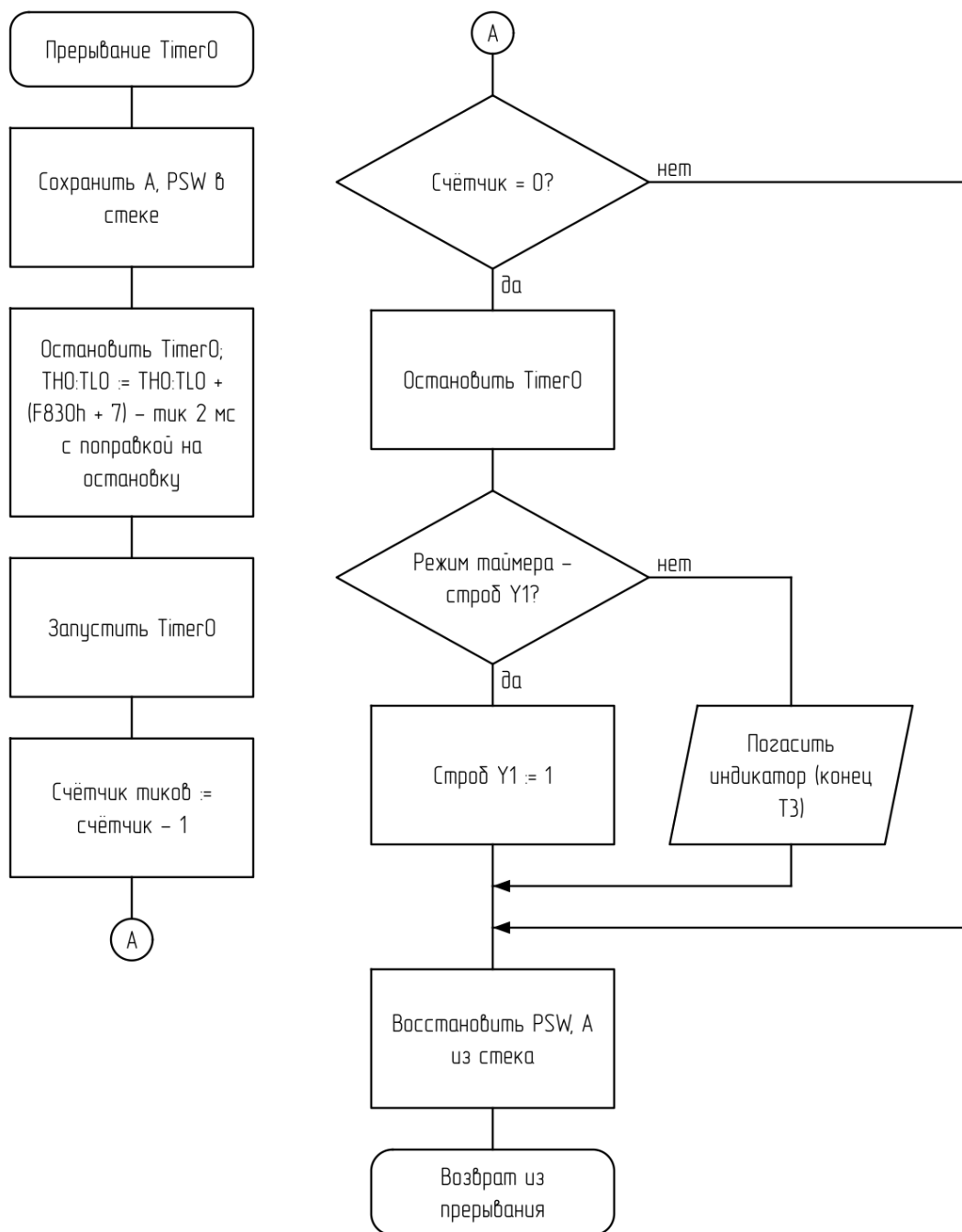


Рисунок 10 — Схема алгоритма: Обработчик прерывания Timer0: отсчёт T1 и T3

Timer0 работает в 16-разрядном режиме и переполняется каждые 2 мс. В обработчике таймер на несколько тактов останавливается, и к тому, что он успел насчитать после переполнения, прибавляется F830h плюс семь тактов остановки — так задержка входа в прерывание не накапливается, и за 88 тиков строб получается 176 мс без заметной ошибки. После перезапуска уменьшается счётчик тиков; когда он доходит до нуля, таймер останавливается и строб Y1 поднимается в 1. A и PSW сохраняются в стеке, потому что прерывание может прийти в любом месте основной программы.

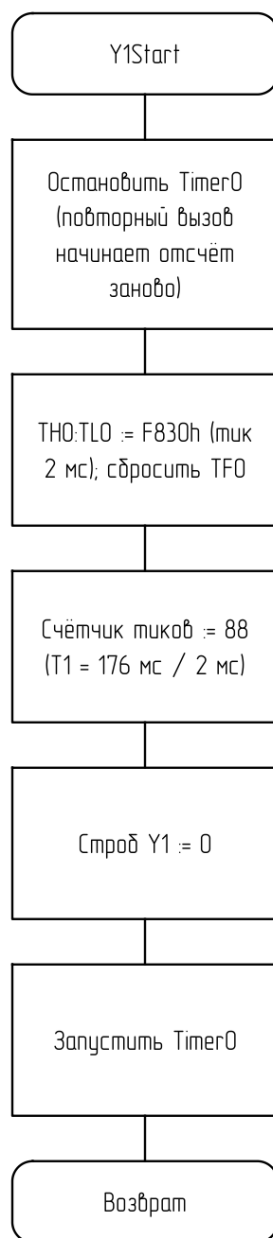


Рисунок 11 — Схема алгоритма: Начало строба Y1

Y1Start не ждёт окончания строба, а только запускает его. Сначала Timer0 останавливается, чтобы повторное нажатие «Тест» начинало отсчёт заново, затем в таймер загружается первый тик 2 мс, а счётчик тиков ставится на 88. После этого вывод P1.1 опускается в 0 и таймер запускается; дальше длительность отсчитывает обработчик Timer0, а процедура сразу возвращается.

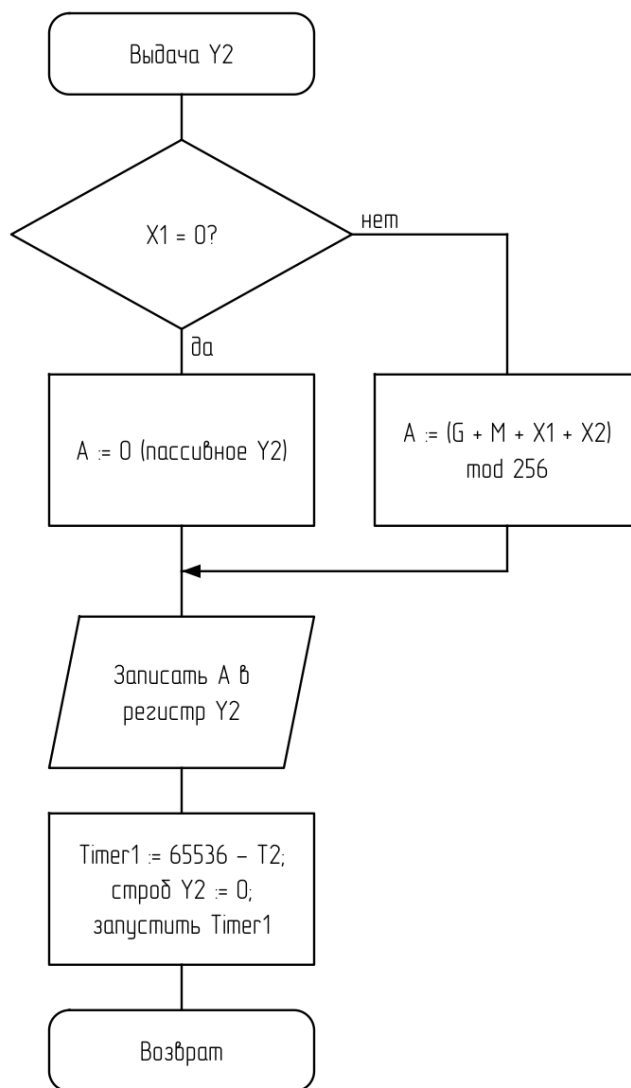


Рисунок 12 — Схема алгоритма: Расчёт и выдача сигнала Y2

Схема «Расчёт и выдача сигнала Y2» выполняется так: вход — «Выдача Y2»; проверяется условие «X1 = 0»: если да — $A := 0$ (пассивное Y2), иначе — $A := (G + M + X1 + X2) \bmod 256$; обмен с устройством: записать A в регистр Y2; $Timer1 := 65536 - T2$; строб Y2 := 0; запустить Timer1; в конце — возврат.

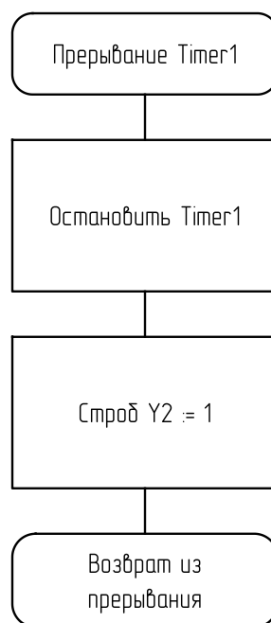


Рисунок 13 — Схема алгоритма: Обработчик прерывания Timer1: конец строба Y2

Схема «Обработчик прерывания Timer1: конец строба Y2» выполняется так: вход — «Прерывание Timer1»; остановить Timer1; строб Y2 := 1; в конце — возврат из прерывания.

3.2 Структура программы с описанием функций составных частей и связи между ними

Программа состоит из инициализации, основной программы (ожидание прерываний) и подпрограмм, описанных ниже. Описание каждой подпрограммы взято из заголовочного комментария в исходном тексте (приложение А).

3.2.1 Подпрограмма KbScan (программа 1)

Назначение: опрос матрицы 3x4 по столбцам (вызывается из обработчика INT0).

Входные данные: нет.

Выходные данные: A — код клавиши (0...9, «Т» = 10, «С» = 11) или 0FFh, если нажато больше одной клавиши или ни одной (помеха).

Используемые регистры: только A; B, DPTR, PSW, R2–R7 сохраняются в стеке.

3.2.2 Подпрограмма BufWrite (программа 2)

Назначение: запись отсчёта в кольцевой буфер IDT7005.

Входные данные: A — отсчёт; HEAD, TAIL, F_EMPTY — текущее состояние буфера.

Выходные данные: HEAD (и при переполнении TAIL) сдвинуты; F_EMPTY = 0; F_OVF = 1 при потере.

Используемые регистры: ничего (A, B, DPTR, PSW сохраняются).

3.2.3 Подпрограмма NextCell (программа 2)

Назначение: следующая ячейка кольцевого буфера.

Входные данные: DPTR — адрес ячейки внутри буфера.

Выходные данные: DPTR — следующая ячейка; после последней (BUF_END - 1) — снова BUF_START.

Используемые регистры: A, флаги.

3.2.4 Подпрограмма Y1Start (программа 3)

Назначение: запуск строба Y1 длительностью T1.

Входные данные: нет.

Выходные данные: PIN_Y1 = 0, Timer0 запущен, Y1Left = Y1_TICKS.

Используемые регистры: ничего (меняет только TH0, TL0, TF0, TR0).

3.2.5 Обработчик прерывания Timer0 — T0Isr (программа 3)

Назначение: обработчик Timer0: прошёл очередной тик 2 мс.

Входные данные: Y1Left.

Выходные данные: Y1Left - 1; при нуле PIN_Y1 = 1 и Timer0 остановлен.

Используемые регистры: ничего (A и PSW в стеке).

Распределение ресурсов микроконтроллера — прерывания, адреса внешних устройств и переменные — приведено в таблицах 1–3.

Таблица 1 — Прерывания

Источник	Вектор	Назначение	Запуск
INT0 (P3.2)	0003h	нажатие клавиши	по спаду
Timer0	000Bh	отсчёт T_1 (176 мс) и T_3 (2150 мс), тик 2 мс	переполнение
INT1 (P3.3)	0013h	строб записи X_2	по спаду
Timer1	001Bh	отсчёт T_2 (1536 мкс)	переполнение
UART	0023h	не используется (заглушка)	—

Таблица 2 — Адреса внешних устройств (обращение командами movx при P3.4 = 1)

Устройство	Выход дешифратора	Адреса
Индикатор	Y2	4000h–5FFFh
Клавиатура	Y4	8000h–9FFFh
Буфер (IDT7005)	Y1	2000h–3FFFh
Регистр Y2	Y3	6000h–7FFFh

Таблица 3 — Переменные во внутреннем ОЗУ

Имя	Адрес	Назначение
HEAD_L, HEAD_H	38h, 39h	«голова» — адрес ячейки буфера для следующей записи
TAIL_L, TAIL_H	3Ah, 3Bh	«хвост» — адрес ячейки буфера для следующего чтения
F_EMPTY	20h.1 (бит 01h)	флаг пустоты буфера
F_OVF	20h.7 (бит 07h)	флаг переполнения буфера
Y1Left	30h	переменная программы 3

3.3 Связи программы с другими программами

Программа «Программа управления объектом» в ходе своей работы не использует других программ.

4 Входные данные

4.1 Клавиатура

Клавиатура — матрица 3×4 (3 столбцов × 4 строк), сканирование по столбцам: в регистр столбцов (адрес 8000h, разряды D0...D2) выдаётся 0 в опрашиваемый столбец, строки читаются через повторитель по тому же адресу (разряды D0...D3, 0 — контакт замкнут). Нажатие любой клавиши при всех столбцах в 0 вызывает прерывание INT0. Коды клавиш (табл. 3 ТЗ) и их расположение — в таблице 4.

Таблица 4 — Расположение клавиш и их коды («клавиша» — код)

	Col1	Col2	Col3
Row1	«1» — 1	«2» — 2	«3» — 3
Row2	«4» — 4	«5» — 5	«6» — 6
Row3	«7» — 7	«8» — 8	«9» — 9
Row4	«0» — 0	«Т» — 10	«С» — 11

Результат опроса — код единственной нажатой клавиши (0–9 — цифры, 10 — «Тест», 11 — «Сброс») или 0FFh, если не нажато ничего или нажато больше одной клавиши. Цифра становится значением X_1 (начальное значение $X_1 = 0$).

4.2 Сигнал X2

X_2 — 8-разрядное целое без знака. Внешнее устройство записывает отсчёт в двухпортовое ОЗУ IDT7005 через свой порт и сопровождает запись стробом, который вызывает прерывание INT1. Отсчёты хранятся в кольцевом буфере на 1210 байт (адреса 2000h–24B9h).

5 Выходные данные

5.1 Характер и организация выходных данных

Индикация. На индикаторе отображается последняя введённая цифра — до ввода следующей, но не дольше $T_3 = 2,150$ с; при ошибке — код ошибки с горящей точкой dp . Начальное состояние — индикатор погашен.

Кольцевой буфер во внешнем ОЗУ. Ячейки 2000h–24B9h. «Голова» — адрес ячейки для следующей записи, «хвост» — для следующего чтения; после последней ячейки следует первая. Буфер пуст — флаг пустоты = 1 (голова = хвост). Если буфер заполнен, новый отсчёт затирает самый старый (хвост сдвигается) и устанавливается флаг переполнения; после чтения флаг переполнения сбрасывается.

Управляющие сигналы. Y_1 — строб низкого уровня длительностью 176 мс по клавише «Тест». $Y_2 = (6 + 1 + X_1 + X_2) \bmod 256$ при $X_1 \neq 0$, иначе $Y_2 = 0$ (пассивное состояние); вырабатывается, пока в буфере есть данные и $X_1 \neq 0$. Каждая запись Y_2 в выходной регистр сопровождается стробом длительностью 1536 мкс; до записи регистр выдаёт предыдущее значение.

Расчётные параметры работы программы. Константы таймеров (кварц 12 МГц, машинный цикл $T_{мц} = 12 / f = 1$ мкс, режим 1 — 16 бит):

ТН1:TL1 = 65536 – $T_2 / T_{мц} = 65536 - 1536 = 64000 = 0FA00h$ (строб Y_2 , таймер 1)

ТН0:TL0 = 65536 – $2 \text{ мс} / T_{мц} = 65536 - 2000 = 63536 = 0F830h$ (тик 2 мс, таймер 0)

$N_{T1} = T_1 / 2 \text{ мс} = 88$; $N_{T3} = T_3 / 2 \text{ мс} = 1075$ (число тиков)

Минимальное время заполнения буфера (при $X_1 = 0$ сигнал Y_2 пассивен и буфер не читается, отсчёты поступают с максимальной интенсивностью):

$t_{\min} = V / f_{X2} = 1210 / 4 = 302,5$ с (5,0 мин).

Максимальная глубина стека для программы 1 — 14 байт (SP от 07h до 15h). Значение получено прогоном программы в симуляторе на всех сценариях проверки (каждая клавиша, одновременные нажатия, помеха на INT0) и складывается из адреса возврата при входе в обработчик INT0 (2 байта), адреса возврата при вызове процедуры опроса (2 байта) и сохраняемых в стеке регистров.

5.2 Формат, описание и способ кодирования выходных данных

Код символа (табл. 2 ТЗ) преобразуется в код сегментов по таблице 5: разряды D0...D7 регистра индикатора соответствуют сегментам a, b, c, d, e, f, g, dp; индикатор с общим катодом — сегмент горит при 1.

Таблица 5 — Коды сегментов индикатора (dp g f e d c b a)

Код	Символ	Двоичный	Шестн.
0	«0»	00111111	3Fh
1	«1»	00000110	06h
2	«2»	01011011	5Bh
3	«3»	01001111	4Fh
4	«4»	01100110	66h
5	«5»	01101101	6Dh
6	«6»	01111101	7Dh
7	«7»	00000111	07h
8	«8»	01111111	7Fh
9	«9»	01101111	6Fh
10	«Е»	01111001	79h
11 и больше	погашен	00000000	00h

Таблица 6 — Коды ошибок (символ с горящей точкой dp, индикация до нажатия любой клавиши, но не дольше T3)

Индикация	Смысл	Когда выставляется
«1.»	нажато несколько клавиш	опрос клавиатуры нашёл больше одного замкнутого контакта
«2.»	нажатие не распознано	прерывание пришло, а замкнутых контактов нет
«3.»	переполнение буфера	запись X_2 затёрла непрочитанный отсчёт
«4.»	повторный «Тест»	«Тест» нажат, пока идёт строб Y_1

Таблица описывает реакцию системы в целом. В моих программах ошибки «1.» и «2.» процедура опроса KbScan не различает — в обоих случаях она возвращает FFh, и обработчик клавиш выводит код ошибки; признаком ошибки «3.» служит флаг переполнения, который ставит BufWrite, когда новый отсчёт затирает непрочитанный, а «4.» относится к повторному нажатию «Тест» во время строба Y_1 (в Y1Start такой вызов просто перезапускает отсчёт T1). Ошибка только показывается на индикаторе с точкой dp; указатели буфера, X_1 и выходы Y_1 , Y_2 при этом не меняются.

Сигнал Y_2 выдаётся 8-разрядным кодом в выходной регистр (адрес 6000h), строб — на выводе P1.2; сигнал Y_1 — на выводе P1.1.

В 16-разрядном режиме (режим 1) таймер 0 перезагружается в обработчике прерывания; задержка входа в прерывание (несколько машинных циклов на тик) компенсируется поправкой к загружаемому значению или учитывается как погрешность длительности. В режиме 2 (8 разрядов с автоперезагрузкой) перезагрузку выполняет сам таймер, а тик набирается программным счётчиком переполнений.

ПЕРЕЧЕНЬ ПРИНЯТЫХ СОКРАЩЕНИЙ

Таблица 7 — Сокращения

Сокращение	Расшифровка
МК	микроконтроллер
МПС	микропроцессорная система
ОЗУ	оперативное запоминающее устройство
ПЗУ	постоянное запоминающее устройство
ЕСПД	единая система программной документации
ТЗ	техническое задание
CS	Chip Select, выбор микросхемы
IRQ	запрос прерывания
SP	указатель стека
DPTR	16-разрядный регистр-указатель данных
dp	десятичная точка индикатора

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]

ПРИЛОЖЕНИЕ А

Листинги программ

Программа 1 — файл «Грошев АВ-код-1»

```
; Грошев А.В., А-06-23, 1, v1
$INCLUDE (vars.inc)          ; адреса, биты, константы варианта; в файл для робота mpcode вклеит его текст
DSEG AT 30h
; переменные в ОЗУ не нужны: всё держится в регистрах банка 0
CSEG AT 0h
    sjmp START
org 03h
    call KbScan ; %proc%
    reti
org 0bh ; "заглушка" для Tmr0_ovf
    nop
    reti
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    setb PIN_CSEN          ; разрешаем дешифратор адреса, иначе клавиатура не выбирается
    mov DPTR, #ADR_KB
    clr A
    movx @DPTR, A          ; опускаем все столбцы: любое нажатие сразу потянет INT0 вниз
    setb IT0               ; INT0 срабатывает по спаду
    clr IE0                ; забываем возможный ложный запрос до разрешения
    setb EX0               ; разрешаем прерывание от клавиатуры
    setb EA                ; и прерывания вообще
    jmp $ ; %stop%

; KbScan — опрос матрицы 3x4 по столбцам (вызывается из обработчика INT0).
; Назначение: найти единственную нажатую клавишу и вернуть её код по табл. 3 ТЗ.
; Вход: нет.
; Выход: А — код клавиши (0...9, «Т» = 10, «С» = 11) или 0FFh, если нажато
;         больше одной клавиши или ни одной (помеха).
; Портит: только А; В, DPTR, PSW, R2-R7 сохраняются в стеке.
KbScan:
    push B
    push PSW
    push DPL
    push DPH
    push AR2
    push AR3
    push AR4
    push AR5
    push AR6
    push AR7
    mov DPTR, #ADR_KB
    mov R2, #0             ; R2 — сколько замкнутых контактов нашли
    mov R3, #0             ; R3 — номер текущего столбца с нуля
    mov R4, #0FEh          ; R4 — маска столбцов: ноль только у опрашиваемого

KsCol:
    mov A, R4
    movx @DPTR, A          ; опускаем один столбец, остальные держим в 1
    movx A, @DPTR          ; снимаем состояние строк
    anl A, #(1 SHL KB_ROWS) - 1 ; оставляем только подключённые строки
    xrl A, #(1 SHL KB_ROWS) - 1 ; теперь 1 — строка замкнута на этот столбец
    mov R5, A              ; R5 — ещё не разобранные замкнутые строки
    mov A, R3
    mov R6, A              ; R6 — индекс клавиши в таблице: строка*KB_COLS + столбец

KsRow:
    mov A, R5
    jz KsNext              ; замкнутых строк в этом столбце больше нет
    clr C
    rrc A                  ; младшая строка уходит в перенос
    mov R5, A
    jnc KsSkip             ; эта строка не замкнута
    inc R2                 ; ещё одно нажатие
    mov A, R6
    mov R7, A              ; запоминаем индекс найденной клавиши

KsSkip:
    mov A, R6
```

```

        add A, #KB_COLS          ; переходим к той же колонке строкой ниже
        mov R6, A
        sjmp KsRow
KsNext:
        mov A, R4
        rl A                     ; ноль маски сдвигается на следующий столбец
        mov R4, A
        inc R3
        cjne R3, #KB_COLS, KsCol
        clr A
        movx @DPTR, A           ; снова все столбцы в 0 – ждём следующего нажатия
        clr IE0                 ; во время опроса линия INT0 дёргалась – лишний запрос не нужен
        cjne R2, #1, KsBad      ; принимаем только одну нажатую клавишу
        mov A, R7
        mov DPTR, #KeyTab
        movc A, @A+DPTR         ; индекс в матрице -> код клавиши
        sjmp KsOut
KsBad:
        mov A, #0FFh           ; ничего не нажато или нажато несколько
KsOut:
        pop AR7
        pop AR6
        pop AR5
        pop AR4
        pop AR3
        pop AR2
        pop DPH
        pop DPL
        pop PSW
        pop B
        ret

; Коды клавиш по рис. 4 ТЗ (3x4): строки сверху вниз, в строке столбцы слева направо
KeyTab:
        DB 1, 2, 3
        DB 4, 5, 6
        DB 7, 8, 9
        DB 0, 10, 11
END

```

Программа 2 — файл «Грошев АВ-код-2»

```

; Грошев А.В., А-06-23, 1, v1
$INCLUDE (vars.inc)          ; адреса, биты, константы варианта; в файл для робота mpscode вклеит его текст
DSEG AT 30h
; переменные не нужны: «голова», «хвост» и флаги уже заданы в vars.inc
CSEG AT 0h
        sjmp START
org 03h ; "заглушка" для INT0
        nop
        reti
org 0bh ; "заглушка" для Tmr0_ovf
        nop
        reti
org 13h ; "заглушка" для INT1
        nop
        reti
org 1bh ; "заглушка" для Tmr1_ovf
        nop
        reti
org 23h ; "заглушка" для UART
        nop
        reti
org 2bh
START:
; Инициализация МК *****
        mov SP, #07h
        setb PIN_CSEN          ; дешифратор включён – буфер доступен по movx
        mov HEAD_L, #LOW(BUF_START) ; буфер пуст: «голова» и «хвост» на первой ячейке
        mov HEAD_H, #HIGH(BUF_START)
        mov TAIL_L, #LOW(BUF_START)
        mov TAIL_H, #HIGH(BUF_START)
        setb F_EMPTY           ; данных нет
        clr F_OVF              ; и ничего не терялось
        mov A, #55h            ; пробный отсчёт для записи
        call BufWrite ; %proc%
        jmp $ ; %stop%

; BufWrite – запись отсчёта в кольцевой буфер IDT7005.
; Назначение: положить байт в ячейку «голова» и сдвинуть «голову»; если буфер
;   полон (голова догнала хвост, буфер не пуст), самый старый отсчёт теряется:
;   «хвост» сдвигается вперёд и ставится флаг переполнения.
; Вход: A – отсчёт; HEAD, TAIL, F_EMPTY – текущее состояние буфера.
; Выход: HEAD (и при переполнении TAIL) сдвинуты; F_EMPTY = 0; F_OVF = 1 при потере.
; Портит: ничего (A, B, DPTR, PSW сохраняются).

```

```

BufWrite:
    push ACC
    push B
    push PSW
    push DPL
    push DPH
    mov B, A                ; отсчёт держим в B, A нужен для сравнений
    jnb F_EMPTY, BwPut      ; пустой буфер полным быть не может
    mov A, HEAD_L
    cjne A, TAIL_L, BwPut    ; голова не на хвосте — место есть
    mov A, HEAD_H
    cjne A, TAIL_H, BwPut    ; места нет: выбрасываем самый старый отсчёт
    mov DPL, TAIL_L
    mov DPH, TAIL_H
    call NextCell
    mov TAIL_L, DPL
    mov TAIL_H, DPH
    setb F_OVF              ; отмечаем, что данные потерялись

BwPut:
    mov DPL, HEAD_L
    mov DPH, HEAD_H
    mov A, B
    movx @DPTR, A           ; кладём отсчёт в ячейку «головы»
    call NextCell
    mov HEAD_L, DPL         ; «голова» смотрит на следующую свободную ячейку
    mov HEAD_H, DPH
    clr F_EMPTY            ; в буфере есть хотя бы один отсчёт
    pop DPH
    pop DPL
    pop PSW
    pop B
    pop ACC
    ret

; NextCell — следующая ячейка кольцевого буфера.
; Вход: DPTR — адрес ячейки внутри буфера.
; Выход: DPTR — следующая ячейка; после последней (BUF_END - 1) — снова BUF_START.
; Портит: A, флаги.
NextCell:
    inc DPTR
    mov A, DPL
    cjne A, #LOW(BUF_END), NcOut ; младший байт не совпал — до конца не дошли
    mov A, DPH
    cjne A, #HIGH(BUF_END), NcOut
    mov DPTR, #BUF_START        ; вышли за буфер — заворачиваем на начало

NcOut:
    ret

END

```

Программа 3 — файл «Грошев АВ-код-3»

```

; Грошев А.В., А-06-23, 1, v1
$INCLUDE (vars.inc)          ; адреса, биты, константы варианта; в файл для работа mpcode вклеит его текст
DSEG AT 30h
Y1Left: DS 1                 ; сколько тиков по 2 мс осталось до конца строга Y1
CSEG AT 0h
    sjmp START
org 03h ; "заглушка" для INT0
    nop
    reti
org 0bh ; Timer0 — тик для отсчёта T1
    ljmp T0Isr
org 13h ; "заглушка" для INT1
    nop
    reti
org 1bh ; "заглушка" для Tmr1_ovf
    nop
    reti
org 23h ; "заглушка" для UART
    nop
    reti
org 2bh
START:
; Инициализация МК *****
    mov SP, #07h
    setb PIN_CSEN            ; дешифратор адреса разрешён
    setb PIN_Y1              ; строб Y1 в пассивном состоянии
    mov Y1Left, #0
    anl TMOD, #0F0h          ; Timer0 — 16-битный счётчик (режим 1), Timer1 не трогаем
    orl TMOD, #01h
    clr TR0                  ; таймер пока стоит
    clr TF0
    setb ET0                 ; прерывание по переполнению Timer0
    setb EA
    call Y1Start ; %proc%

```

```

        jmp $ ; %stop%

; Y1Start – запуск строба Y1 длительностью T1.
; Назначение: опустить PIN_Y1 в 0 и запустить Timer0 на Y1_TICKS тиков по 2 мс;
; строб поднимает обработчик Timer0, процедура сразу возвращается.
; Вход: нет. Выход: PIN_Y1 = 0, Timer0 запущен, Y1Left = Y1_TICKS.
; Портит: ничего (меняет только TH0, TL0, TF0, TR0).
Y1Start:
        clr TR0                      ; повторный вызов начинает отсчёт заново
        mov TH0, #TICK_H              ; первый тик – ровно 2 мс
        mov TL0, #TICK_L
        clr TF0
        mov Y1Left, #Y1_TICKS
        clr PIN_Y1                    ; начало строба
        setb TR0
        ret

; T0Isr – обработчик Timer0: прошёл очередной тик 2 мс.
; Назначение: перезарядить Timer0 с учётом тактов, натаивших после переполнения,
; уменьшить счётчик и по его обнулению закончить строб Y1.
; Вход: Y1Left. Выход: Y1Left - 1; при нуле PIN_Y1 = 1 и Timer0 остановлен.
; Портит: ничего (A и PSW в стеке).
T0Isr:
        push ACC
        push PSW
        clr TR0                      ; на время перезарядки таймер стоит (7 тактов)
        mov A, TL0
        add A, #LOW((TICK_H SHL 8) + TICK_L + 7) ; добавляем к уже натаившему и
        mov TL0, A                    ; учитываем 7 тактов остановки
        mov A, TH0
        addc A, #HIGH((TICK_H SHL 8) + TICK_L + 7)
        mov TH0, A
        setb TR0
        djnz Y1Left, T0Out            ; строб ещё идёт
        clr TR0                        ; T1 истекло: таймер больше не нужен
        setb PIN_Y1                    ; конец строба
T0Out:
        pop PSW
        pop ACC
        reti
END

```